

GymTrack Pro

Gestión de Rutinas y Progresión de Cargas

Memoria del Proyecto Integrado (P.I.)

Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web

Autor: Diego Cabello Bautista

Tutor del Proyecto: Andrés Herrera Herrera

Centro Educativo: IES Politécnico Hermenegildo Lanz

Asignatura: Proyecto Integrado (P.I.)

Curso Académico: 2025 – 2026

Índice de Contenidos

Contenido

Índice de Contenidos.....	2
1. Introducción – Análisis del Problema	5
1.1. Contexto del sector fitness actual.....	5
1.2. El problema del entrenamiento no estructurado.....	5
1.3. La sobrecarga progresiva como principio fundamental del entrenamiento de fuerza	6
1.4. Carencias en las soluciones existentes en el mercado	7
1.5. Justificación del proyecto.....	8
2. Objetivos	8
2.1. Objetivo General	8
2.2. Objetivos Específicos.....	8
Objetivos de Funcionalidad	9
Objetivos de Arquitectura e Infraestructura	9
Objetivos de Calidad y Documentación.....	10
3. Funciones y Rendimientos Deseados	10
3.1. Definición del Producto Mínimo Viable (MVP).....	10
3.2. Módulo de Autenticación y Gestión de Usuarios.....	11
3.3. Constructor de Rutinas (CRUD Completo)	11
3.4. Registro de Marcas en Tiempo Real	12
3.5. Motor de Visualización y Gráficas Dinámicas	13
3.6. Diseño Responsive y Experiencia de Usuario en Gimnasio	13
3.7. Infraestructura y Despliegue Contenerizado	14
3.8. Requisitos de Rendimiento No Funcionales	14
4. Planteamiento de la Solución y Justificación.....	15
4.1. Arquitectura general de la solución.....	15
4.2. Justificación del stack tecnológico	16
4.3. React.js como framework de frontend	16

4.4.	PHP como lenguaje de backend	17
4.5.	MySQL / MariaDB como sistema de base de datos.....	18
4.6.	Docker y Nginx como infraestructura de despliegue	18
5.	Modelado.....	19
5.1.	Diagrama entidad-relacion	19
5.2.	Diagrama de casos de uso	19
5.3.	Diagrama de flujo de la aplicación.....	20
6.	Recursos	21
6.1.	Recursos Hardware	21
6.2.	Recursos Software.....	22
6.3.	Recursos Humanos.....	22
7.	Recursos	23
8.	Recursos	24
8.1.	Diseño de la Base de Datos.....	24
8.2.	Arquitectura del backend (API REST en PHP).....	24
8.3.	Implementación del frontend con React.js.....	24
8.4.	Configuración de Docker y Nginx	25
8.5.	Sistema de autenticación con JWT.....	25
8.6.	Motor de gráficas y visualización de datos	26
9.	Fase de pruebas	26
9.1.	Estrategia de pruebas.....	26
9.2.	Pruebas unitarias del backend	26
9.3.	Pruebas de integración de la API.....	26
9.4.	Pruebas de interfaz y usabilidad.....	27
9.5.	Pruebas de rendimiento y carga	27
9.6.	Pruebas de seguridad.....	27
10.	 Documentación	27
11.	 Conclusión	28

12.....	Bibliografia
.....	30

.....

1. Introducción – Análisis del Problema

1.1. Contexto del sector fitness actual

En las últimas décadas, la práctica del ejercicio físico ha experimentado una transformación profunda tanto en términos cualitativos como cuantitativos. Según datos del Eurobarómetro del Deporte y la Actividad Física (edición 2022), el número de europeos que practican actividad física de forma regular ha aumentado de manera sostenida, impulsado por una mayor conciencia sobre los beneficios del ejercicio para la salud física y mental. España, en particular, registra un crecimiento notable en la matriculación en gimnasios y en el consumo de contenidos deportivos digitales, tendencia que se aceleró significativamente tras la pandemia de COVID-19, cuando el entrenamiento en casa y las aplicaciones de fitness experimentaron picos históricos de descarga.

Este auge ha dado lugar a un ecosistema tecnológico de apoyo al deporte de enorme dinamismo. Aplicaciones de running como Strava, plataformas de fitness como Nike Training Club, o wearables de marcas como Garmin y Fitbit han democratizado la monitorización del rendimiento atlético. Sin embargo, el segmento específico del entrenamiento de fuerza —musculación, powerlifting, CrossFit, halterofilia— presenta particularidades que las soluciones generalistas no abordan con suficiente profundidad técnica.

El entrenamiento de fuerza requiere un control meticuloso de variables que van más allá de la distancia recorrida o las calorías quemadas: el peso levantado por ejercicio, el número de repeticiones y series, el tiempo de descanso entre series, el volumen total de entrenamiento semanal y la progresión de la carga a lo largo del tiempo son métricas fundamentales que determinan la eficacia del programa de entrenamiento. Esta naturaleza multidimensional del dato de fuerza hace que las aplicaciones de fitness genéricas resulten insuficientes para el deportista serio que busca maximizar su rendimiento mediante metodología científica.

1.2. El problema del entrenamiento no estructurado

La gran mayoría de los practicantes de musculación, tanto amateurs como intermedios, incurren en lo que los especialistas en ciencias del deporte denominan "entrenamiento intuitivo no documentado". Este fenómeno consiste en acudir al gimnasio sin un plan de trabajo predefinido, seleccionar ejercicios de manera relativamente aleatoria, y no registrar los pesos, las repeticiones ni el volumen total completado en cada sesión. Las consecuencias de este enfoque son predecibles y documentadas en la literatura científica del entrenamiento:

- **Estancamiento prematuro:** Sin un registro histórico, el deportista no puede determinar con precisión si ha progresado, se ha estancado o ha regresado en sus marcas. La percepción subjetiva del esfuerzo es un indicador notoriamente poco fiable de la progresión real.
- **Sobreentrenamiento inadvertido:** La acumulación excesiva de volumen sin monitorización puede llevar al síndrome de sobreentrenamiento, caracterizado por fatiga crónica, descenso del rendimiento y mayor susceptibilidad a lesiones.
- **Infraentrenamiento crónico:** El fenómeno opuesto es igualmente común. Muchos deportistas realizan sistemáticamente menos volumen e intensidad del que su nivel de recuperación y adaptación podría tolerar, perdiendo oportunidades de mejora.
- **Pérdida de motivación:** La ausencia de datos objetivos de progresión elimina uno de los principales motivadores del entrenamiento a largo plazo. Ver graficada la propia curva de progreso —ese 2,5 kg extra en el press de banca que hace tres meses parecía imposible— tiene un valor psicológico y motivacional enorme.
- **Incapacidad de replicar resultados positivos:** Cuando ocasionalmente se produce una sesión excepcionalmente buena, la falta de registro impide identificar qué variables contribuyeron a ese rendimiento superior para reproducirlo de manera sistemática.

Este problema no es exclusivo de los deportistas aficionados. Incluso atletas con años de experiencia tienden a subestimar o sobreestimar los pesos empleados en sesiones pasadas cuando no cuentan con registros objetivos. La memoria humana está sujeta a sesgos cognitivos que distorsionan el recuerdo del rendimiento deportivo, haciendo que la documentación rigurosa sea prácticamente indispensable para la mejora continua.

1.3. La sobrecarga progresiva como principio fundamental del entrenamiento de fuerza

El principio de sobrecarga progresiva es uno de los pilares más sólidamente establecidos de la fisiología del ejercicio. Enunciado en su forma moderna por el Dr. Thomas DeLorme en la década de 1940 a partir de sus trabajos con veteranos de guerra, establece que, para que el organismo continúe adaptándose y mejorando su capacidad de producción de fuerza, es necesario que los estímulos de entrenamiento aumenten de manera gradual y sistemática a lo largo del tiempo.

Esta progresión puede materializarse a través de múltiples mecanismos: incrementando el peso absoluto levantado, aumentando el número de repeticiones con el mismo peso, añadiendo series

adicionales, reduciendo los períodos de descanso, mejorando la velocidad de ejecución o introduciendo variaciones que aumenten la dificultad del movimiento. En cualquiera de sus formas, la sobrecarga progresiva requiere de un punto de referencia claro: el dato del rendimiento anterior.

Sin ese dato, la progresión se convierte en aleatoria. El deportista no sabe desde dónde partir, hacia dónde avanzar ni a qué ritmo debería hacerlo. Aplicar sobrecarga progresiva sin registro es como intentar navegar sin brújula: es posible avanzar en alguna dirección, pero resulta extremadamente ineficiente y raramente se llega al destino deseado de la manera más óptima.

GymTrack Pro nace precisamente para resolver esta carencia, proporcionando la infraestructura digital que hace posible aplicar el principio de sobrecarga progresiva de forma rigurosa, objetiva y continuada en el tiempo.

1.4. Carencias en las soluciones existentes en el mercado

El mercado actual de aplicaciones de seguimiento del entrenamiento de fuerza no está vacío. Existen productos como Strong, Hevy, JEFIT o el módulo de pesas de Fitbod, cada uno con sus propias fortalezas y propuesta de valor. Sin embargo, un análisis crítico de estas soluciones revela limitaciones significativas que justifican el desarrollo de una alternativa propia:

- **Barrera de pago (Freemium agresivo):** La mayoría de aplicaciones líderes del mercado restringen las funcionalidades más valiosas —en particular las gráficas de progresión y el historial ilimitado— a planes de pago de coste mensual recurrente. Esto representa una barrera de acceso real para el deportista joven o el estudiante con recursos limitados.
- **Complejidad de interfaz:** Aplicaciones como JEFIT ofrecen una cantidad abrumadora de funcionalidades que generan una curva de aprendizaje elevada y una experiencia de usuario fragmentada. En el contexto del gimnasio, donde el tiempo entre series es limitado, la interfaz debe ser intuitiva y de respuesta inmediata.
- **Dependencia de ecosistemas propietarios:** Muchas aplicaciones están diseñadas en torno a bases de datos de ejercicios predefinidas y no permiten la personalización completa de los ejercicios, los parámetros de registro ni la estructura de las rutinas.
- **Ausencia de visualización de datos significativa:** Las gráficas de progresión, cuando existen, suelen ser simplistas y no permiten el análisis comparativo entre períodos o ejercicios con la granularidad que un deportista técnico necesita.
- **Falta de control sobre los propios datos:** Al utilizar servicios en la nube de terceros, el usuario pierde el control efectivo sobre sus datos de rendimiento, quedando sujeto a las políticas de privacidad y comerciales de empresas cuya actividad principal no es precisamente la gestión ética de la información personal.

1.5. Justificación del proyecto

La convergencia de los factores descritos —el crecimiento del sector fitness, el problema endémico del entrenamiento no documentado, la importancia científicamente validada de la sobrecarga progresiva y las carencias de las soluciones comerciales existentes— configura un espacio de oportunidad claro para el desarrollo de GymTrack Pro.

Este proyecto propone una solución técnicamente sólida, completamente open-source en su lógica de negocio, desplegable en infraestructura propia mediante contenedores Docker, y centrada en ofrecer la mejor experiencia posible en el caso de uso específico del registro y análisis del entrenamiento de fuerza. No aspira a competir con los grandes ecosistemas de fitness generalista, sino a resolver con máxima eficacia el problema concreto del deportista que quiere tomar decisiones de entrenamiento basadas en datos objetivos.

Desde el punto de vista académico, el proyecto representa además una oportunidad de integrar de forma coherente y práctica las competencias adquiridas a lo largo del ciclo formativo: desarrollo frontend moderno con React.js, diseño e implementación de APIs REST en PHP, modelado y administración de bases de datos relacionales con MySQL/MariaDB, infraestructura de despliegue con Docker y Nginx, y control de versiones profesional con Git y GitHub.

2. Objetivos

2.1. Objetivo General

El objetivo general del presente proyecto es **diseñar, desarrollar y desplegar una aplicación web de gestión del entrenamiento de fuerza** —GymTrack Pro— que permita a los deportistas registrar sus rutinas, anotar los pesos y repeticiones realizados en cada sesión, y visualizar de forma dinámica la progresión de su rendimiento a lo largo del tiempo, utilizando para ello un stack tecnológico moderno, escalable y contenerizado.

Esta aplicación deberá ser funcional en su versión mínima viable (MVP), accesible desde dispositivos móviles, segura en la gestión de credenciales de usuario, y susceptible de ser ampliada con funcionalidades adicionales en fases de desarrollo posteriores.

2.2. Objetivos Específicos

Con el fin de operacionalizar el objetivo general en metas concretas, medibles y alcanzables dentro del marco temporal del proyecto, se definen los siguientes objetivos específicos:

Objetivos de Funcionalidad

1. **Implementar un sistema de autenticación seguro** que permita el registro de nuevos usuarios mediante correo electrónico y contraseña, el inicio de sesión con generación de tokens JWT, y la gestión segura de la sesión del usuario en el cliente.
2. **Desarrollar un módulo CRUD completo para la gestión de rutinas** que permita al usuario crear planes de entrenamiento personalizados, estructurados en días y ejercicios, con la posibilidad de editarlos y eliminarlos en cualquier momento.
3. **Construir un sistema de registro de marcas en tiempo real** que facilite la anotación de pesos, repeticiones y series durante la ejecución de la sesión de entrenamiento, con acceso inmediato al historial de la sesión anterior para el mismo ejercicio.
4. **Integrar un motor de visualización de datos** basado en gráficas dinámicas e interactivas que represente la evolución del rendimiento por ejercicio, mostrando la progresión de carga máxima, volumen total y otras métricas relevantes a lo largo de las sesiones registradas.
5. **Garantizar el diseño responsive de la interfaz** para que la aplicación sea completamente usable en smartphones, tablets y ordenadores de escritorio, priorizando la experiencia de uso en condiciones de gimnasio (una mano ocupada, pantalla pequeña, sesión activa).

Objetivos de Arquitectura e Infraestructura

1. **Diseñar e implementar una API REST** en PHP que exponga los recursos de la aplicación de forma semántica, siguiendo las convenciones REST estándar (métodos HTTP, códigos de estado, formato JSON), y que sea consumida de manera eficiente por el cliente React.
2. **Modelar una base de datos relacional** en MySQL/MariaDB que soporte todas las entidades del dominio del problema (usuarios, rutinas, ejercicios, sesiones, marcas) con las relaciones e integridades referenciales apropiadas.
3. **Contenerizar la aplicación completa mediante Docker**, definiendo servicios independientes para el frontend, el backend, la base de datos y el servidor web Nginx, orquestados a través de un archivo `docker-compose.yml` que facilite el despliegue reproducible del entorno en cualquier máquina anfitrión.
4. **Configurar Nginx como servidor web y proxy inverso** para gestionar el enrutamiento de peticiones hacia el servidor PHP y el cliente React, habilitando la compresión GZIP, los encabezados de caché apropiados y la redirección HTTPS.

5. **Establecer un flujo de trabajo profesional con Git y GitHub**, utilizando ramas de desarrollo, commits semánticos descriptivos y, en la medida de lo posible, pull requests que documenten los cambios incrementales del proyecto.

Objetivos de Calidad y Documentación

1. **Aplicar buenas prácticas de seguridad** en el desarrollo, incluyendo la protección contra SQL Injection mediante sentencias preparadas (PDO), la validación de entradas tanto en el cliente como en el servidor, el hashado seguro de contraseñas con `bcrypt`, y la implementación correcta de cabeceras CORS.
2. **Realizar una fase de pruebas documentada** que cubra las funcionalidades críticas del sistema, con casos de prueba definidos para los módulos de autenticación, gestión de rutinas, registro de marcas y visualización de datos.
3. **Identificar y documentar las funcionalidades de extensión futura** (trabajo futuro), describiéndolas con suficiente detalle técnico como para que sirvan de base a futuros desarrollos incrementales sobre el proyecto.
4. **Elaborar la presente memoria técnica** conforme a los estándares de documentación exigidos para el Proyecto Integrado del ciclo formativo, incluyendo análisis del problema, justificación de decisiones de diseño, diagramas de modelado, descripción de la implementación, resultados de pruebas y conclusiones.

3. Funciones y Rendimientos Deseados

3.1. Definición del Producto Mínimo Viable (MVP)

En el contexto del desarrollo ágil de software, un **Producto Mínimo Viable** (MVP, por sus siglas en inglés: *Minimum Viable Product*) se define como la versión más reducida de un producto que permite ofrecer valor real al usuario final y, simultáneamente, obtener retroalimentación válida para orientar el desarrollo futuro. El concepto, popularizado por Eric Ries en el marco de la metodología Lean Startup, implica priorizar aquellas funcionalidades sin las cuales el producto carece de razón de ser, eliminando o posponiendo todo aquello que, siendo deseable, no resulta imprescindible para la propuesta de valor central.

En el caso de GymTrack Pro, el MVP se define como el conjunto mínimo de funcionalidades que habilita al deportista para **registrar una sesión de entrenamiento completa y consultar la progresión de sus marcas**. Cualquier funcionalidad que no sea estrictamente necesaria para cumplir este ciclo básico queda excluida del MVP y se documenta como trabajo futuro.

A continuación se detallan las funciones que conforman el MVP, junto con los requisitos de rendimiento y criterios de aceptación asociados a cada una de ellas.

3.2. Módulo de Autenticación y Gestión de Usuarios

El sistema de autenticación constituye la base sobre la que se sustenta la naturaleza personalizada de la aplicación. Cada usuario debe poder crear y acceder a su propio espacio de datos de entrenamiento de forma segura y privada.

Funciones requeridas:

- Registro de nuevos usuarios mediante formulario con nombre, dirección de correo electrónico y contraseña. La contraseña deberá cumplir requisitos mínimos de complejidad (longitud mínima de 8 caracteres).
- Inicio de sesión mediante correo electrónico y contraseña, con generación y entrega al cliente de un token JWT (JSON Web Token) firmado con una clave secreta almacenada en variables de entorno.
- Persistencia de la sesión en el navegador mediante el almacenamiento del token en `localStorage` o en una cookie `HttpOnly`, con un tiempo de expiración configurable.
- Cierre de sesión (logout) que invalide el token en el cliente.
- Protección de todas las rutas de la API que requieran identificación mediante validación del token JWT en cada petición.

Requisitos de rendimiento: El proceso de autenticación (inicio de sesión) deberá completarse en un tiempo inferior a 1 segundo en condiciones normales de red. La validación del token JWT en las peticiones autenticadas no deberá añadir una latencia perceptible al tiempo de respuesta de la API.

3.3. Constructor de Rutinas (CRUD Completo)

El módulo de rutinas es el núcleo estructural de la aplicación. Permite al usuario definir los planes de entrenamiento que va a seguir, organizando los ejercicios que realizará en cada sesión.

Funciones requeridas:

- **Crear (Create):** El usuario podrá crear una nueva rutina asignándole un nombre descriptivo. Dentro de cada rutina, podrá añadir ejercicios personalizados indicando su nombre y, opcionalmente, notas adicionales (grupo muscular, tipo de movimiento, etc.).
- **Leer (Read):** El usuario dispondrá de un listado de todas sus rutinas activas, con la posibilidad de acceder al detalle de cada una para ver los ejercicios que la componen.
- **Actualizar (Update):** Tanto el nombre de la rutina como los ejercicios que la componen podrán ser modificados en cualquier momento, añadiendo o eliminando ejercicios según las necesidades del programa de entrenamiento.
- **Eliminar (Delete):** El usuario podrá eliminar una rutina completa. La eliminación requerirá una confirmación explícita por parte del usuario para evitar eliminaciones accidentales. El comportamiento de los registros de sesiones asociados a la rutina eliminada deberá definirse (conservación o eliminación en cascada).

Requisitos de rendimiento: Las operaciones de listado de rutinas deberán completarse en menos de 500 ms. Las operaciones de creación, actualización y eliminación deberán proporcionar retroalimentación visual al usuario en menos de 300 ms.

3.4. Registro de Marcas en Tiempo Real

Este módulo representa el corazón funcional de GymTrack Pro y el diferenciador clave frente al entrenamiento no documentado. Permite al usuario anotar los datos de cada serie durante la sesión de entrenamiento activa.

Funciones requeridas:

- Selección de la rutina a ejecutar e inicio de una nueva sesión de entrenamiento.
- Para cada ejercicio de la rutina, registro de cada serie realizada indicando el peso utilizado (en kilogramos, con soporte para decimales como 0.5 kg) y el número de repeticiones completadas.
- Visualización inmediata, en la misma pantalla de registro, del peso y las repeticiones utilizadas en la sesión anterior para el mismo ejercicio, a modo de referencia de progresión.
- Posibilidad de añadir series adicionales no previstas inicialmente o de omitir ejercicios de la rutina si fuera necesario.
- Guardado automático o manual de la sesión al finalizarla, registrando la fecha y hora de la misma.

- Visualización del historial de sesiones completadas, ordenadas cronológicamente de forma descendente.

Requisitos de rendimiento: El registro de una serie individual (envío del formulario y confirmación visual) deberá completarse en menos de 500 ms. La aplicación deberá ser completamente funcional con una sola mano en un smartphone de pantalla estándar (5–6 pulgadas), dado que la otra mano puede estar ocupada durante el entrenamiento.

3.5. Motor de Visualización y Gráficas Dinámicas

El módulo de visualización transforma los datos brutos de marcas en información accionable. Es el componente que cierra el ciclo analítico de la aplicación y proporciona el mayor valor diferencial al deportista.

Funciones requeridas:

- Gráfica de evolución de la **carga máxima** por sesión para cada ejercicio, representada como una línea temporal que muestra el mejor registro (mayor peso en cualquier número de repeticiones) de cada sesión.
- Gráfica de **volumen total** por sesión, calculado como la suma del producto peso $\times$ repeticiones para todas las series del ejercicio en cada sesión.
- Interactividad básica de las gráficas: hover para mostrar el valor exacto del punto, zoom sobre el eje temporal, y posibilidad de seleccionar el ejercicio a visualizar mediante un menú desplegable.
- Tabla resumen con las últimas N sesiones registradas para un ejercicio dado, mostrando fecha, series totales, peso máximo y volumen total.

Requisitos de rendimiento: Las gráficas deberán renderizarse en menos de 1 segundo para historiales de hasta 100 sesiones. La biblioteca de gráficas seleccionada deberá ser ligera y no impactar negativamente en el tiempo de carga inicial de la aplicación.

3.6. Diseño Responsive y Experiencia de Usuario en Gimnasio

La naturaleza del caso de uso —un deportista usando la aplicación en un gimnasio, con posibles condiciones de iluminación adversas, las manos sudadas y el tiempo limitado entre series— impone requisitos de usabilidad específicos que deben guiar cada decisión de diseño de interfaz.

Funciones y requisitos de diseño:

- Interfaz completamente adaptativa a los anchos de pantalla más comunes: smartphones (320–480px), tablets (768–1024px) y escritorio (1280px+).
- Elementos de interacción (botones, campos de formulario) con áreas táctiles mínimas de 44×44 píxeles, conforme a las directrices de accesibilidad WCAG 2.1.
- Contraste de colores suficiente para la legibilidad en condiciones de iluminación variada, con soporte para modo oscuro como mejora deseable.
- Flujos de interacción mínimos: el número de taps/clics necesarios para registrar una serie desde la pantalla de sesión activa no deberá superar tres.
- Retroalimentación visual inmediata (estado de carga, confirmaciones, mensajes de error) en todas las operaciones que impliquen comunicación con el servidor.

3.7. Infraestructura y Despliegue Contenerizado

La infraestructura de despliegue es un componente técnico de primer orden que determina la reproducibilidad, la escalabilidad y el coste de operación de la aplicación. GymTrack Pro adopta una arquitectura basada en contenedores Docker como estándar de despliegue.

Funciones requeridas de la infraestructura:

- Definición de al menos tres servicios Docker diferenciados: servidor web Nginx, servidor de aplicación PHP-FPM, y servidor de base de datos MariaDB.
- Orquestación de los servicios mediante `docker-compose.yml`, con definición de redes internas, volúmenes persistentes para los datos de la base de datos, y variables de entorno para la configuración sensible.
- El servicio de frontend React, una vez compilado a archivos estáticos, será servido directamente por Nginx.
- Script de inicialización de la base de datos que cree automáticamente el esquema de tablas y los datos semilla necesarios al levantar el entorno por primera vez.

3.8. Requisitos de Rendimiento No Funcionales

Más allá de los requisitos funcionales asociados a cada módulo, el sistema en su conjunto debe satisfacer los siguientes requisitos de rendimiento no funcionales:

Categoría	Requisito	Criterio de Aceptación
Tiempo de carga inicial	La aplicación debe cargarse rápidamente en la primera visita	Tiempo de carga (LCP) inferior a 2,5 segundos en conexión 4G
Disponibilidad	El servicio debe estar disponible de forma continua	Uptime objetivo del 99% en entorno de producción
Seguridad de datos	Las contraseñas no deben almacenarse en texto plano	Uso de bcrypt con coste mínimo de 10 rondas
Protección API	Los endpoints privados deben requerir autenticación	Respuesta 401 Unauthorized para peticiones sin token válido
Consistencia de datos	Las operaciones sobre la BD deben ser atómicas cuando proceda	Uso de transacciones SQL en operaciones de escritura compuestas
Usabilidad móvil	Operabilidad completa con una sola mano	Elementos táctiles mínimos de 44x44px, flujos de máximo 3 taps

4. Planteamiento de la Solución y Justificación

4.1. Arquitectura general de la solución

El sistema de GymTrack Pro se ha diseñado siguiendo un patrón de arquitectura **Cliente-Servidor** estructurado en tres capas claramente diferenciadas. Esta separación de responsabilidades (*Separation of Concerns*) garantiza un código más limpio, mantenible y preparado para futuras ampliaciones.

1. **Capa de Presentación (Frontend):** Constituye la interfaz de usuario con la que interactúa el deportista. Se ha desarrollado como una *Single Page Application* (SPA), lo que significa que la carga inicial descarga los recursos estáticos necesarios y, a partir de ese momento, la navegación entre pantallas y componentes se realiza sin necesidad de recargar la página completa, ofreciendo una experiencia fluida casi idéntica a la de una aplicación nativa móvil.

2. **Capa de Lógica de Negocio (Backend):** Actúa como el motor del sistema. Se expone a través de una API RESTful que recibe las peticiones HTTP (GET, POST, PUT, DELETE) enviadas desde el frontend, procesa la lógica correspondiente (autenticación de usuarios, cálculos de volumen, validación de datos) y devuelve respuestas estructuradas en formato JSON.
3. **Capa de Persistencia de Datos (Base de Datos):** Es la encargada de almacenar de forma permanente y segura toda la información generada en la plataforma: credenciales, catálogos de ejercicios, rutinas personalizadas y el histórico de marcas de cada sesión.

Toda esta arquitectura opera dentro de un ecosistema contenerizado, lo que aísla los procesos y asegura que el software funcione idénticamente en el entorno de desarrollo local y en el servidor de producción.

4.2. Justificación del stack tecnológico

La elección del conjunto de tecnologías (*stack* tecnológico) para GymTrack Pro no ha sido arbitraria, sino el resultado de evaluar múltiples factores: las necesidades del Producto Mínimo Viable (MVP), la curva de aprendizaje de la asignatura de Proyecto Integrado, los recursos disponibles y los estándares actuales de la industria del desarrollo web.

Se ha optado por una combinación de tecnologías robustas, ampliamente documentadas y con grandes comunidades de soporte. Frente a soluciones monolíticas más antiguas, la separación estricta entre un frontend moderno impulsado por JavaScript y un backend clásico permite un flujo de trabajo ágil. Además, el uso de herramientas *Open Source* (código abierto) elimina los costes de licenciamiento, haciendo el proyecto viable económicamente. A continuación, se desgranar y justifican individualmente cada uno de los elementos seleccionados.

4.3. React.js como framework de frontend

Para la construcción de la interfaz de usuario se ha seleccionado **React.js**, una librería de JavaScript desarrollada por Meta (Facebook). Su elección se fundamenta en los siguientes pilares técnicos:

- **Arquitectura basada en componentes:** React permite dividir la interfaz de usuario (UI) en piezas pequeñas, independientes y reutilizables (por ejemplo: un botón de "Añadir Serie", una tarjeta de "Rutina" o una gráfica de progreso). Esto no solo acelera el desarrollo, sino que facilita enormemente el mantenimiento y la consistencia visual de la aplicación.
- **Virtual DOM y Rendimiento:** Uno de los mayores cuellos de botella en aplicaciones web complejas es la manipulación directa del DOM (Document Object Model) del navegador. React utiliza un *Virtual DOM*, una representación en memoria del DOM real. Cuando los

datos cambian (por ejemplo, cuando el usuario anota un nuevo peso), React calcula la diferencia de forma ultra-rápida y actualiza únicamente el nodo necesario en el navegador, garantizando un rendimiento óptimo en dispositivos móviles con recursos limitados.

- **Gestión del Estado:** En una aplicación como GymTrack Pro, donde la información cambia constantemente durante un entrenamiento (temporizadores, repeticiones completadas, cargas levantadas), la gestión del estado es crítica. El ecosistema de React, junto con sus *Hooks* (useState, useEffect, useContext), permite un control granular y eficiente de este flujo de datos.
- **Enfoque Mobile-First:** Dado que el entorno principal de uso de la aplicación será el interior de un gimnasio, la UI se ha diseñado priorizando la visualización en teléfonos móviles. React facilita la integración con frameworks de estilos responsivos (como Tailwind CSS o librerías de componentes), asegurando que los elementos táctiles sean accesibles y la información sea legible en pantallas reducidas.

4.4. PHP como lenguaje de backend

En el lado del servidor, la lógica de negocio y la API RESTful han sido programadas utilizando **PHP**. Aunque en el mercado existen alternativas modernas como Node.js, Python (Django) o Java (Spring), la elección de PHP está plenamente justificada para este proyecto:

- **Madurez y Estabilidad:** PHP es un lenguaje veterano que propulsa una inmensa parte de la web actual. Su robustez y estabilidad están sobradamente demostradas.
- **Eficiencia en el despliegue de APIs:** PHP ha evolucionado significativamente en sus últimas versiones (PHP 8+), mejorando drásticamente su rendimiento, gestión de memoria y tipado de datos. Es un lenguaje perfectamente capacitado para manejar el enrutamiento HTTP, la recepción de *payloads* JSON y la validación de peticiones de manera rápida y sin exceso de configuraciones.
- **Integración nativa:** La sincronización entre PHP y bases de datos relacionales es excepcionalmente fluida. A través de extensiones seguras como PDO (*PHP Data Objects*), se pueden realizar consultas parametrizadas que mitigan eficazmente los riesgos de ataques por inyección SQL, un requisito indispensable para salvaguardar los datos personales de los deportistas.

4.5. MySQL / MariaDB como sistema de base de datos

La gestión de la persistencia de los datos se ha confiado a un Sistema de Gestión de Bases de Datos Relacionales (RDBMS), concretamente **MySQL / MariaDB**.

- **Naturaleza Relacional de los Datos:** La información que maneja GymTrack Pro es estrictamente estructurada e interdependiente. Un "Usuario" tiene "Rutinas", una "Rutina" está compuesta de "Ejercicios", y cada ejercicio registra múltiples "Series" en una "Sesión" concreta. Este modelo de relaciones (1 a N, N a M) se modela de forma natural y eficiente mediante tablas, claves primarias y claves foráneas, garantizando la integridad referencial.
- **Cumplimiento ACID:** MySQL asegura transacciones con propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Esto significa que, si se interrumpe la conexión justo cuando el usuario está guardando su entrenamiento, la base de datos no quedará en un estado corrupto o inconsistente.
- **Escalabilidad y Rendimiento:** Las consultas SQL (estructuradas mediante *JOINS* e índices adecuados) permiten recuperar complejos históricos de entrenamiento en fracciones de segundo, siendo el pilar fundamental para alimentar el motor de visualización y las gráficas del frontend de manera casi instantánea.

4.6. Docker y Nginx como infraestructura de despliegue

El despliegue de software tradicional suele enfrentar el clásico problema de "*en mi máquina funciona*". Para erradicar este inconveniente y profesionalizar el ciclo de vida de la aplicación, la infraestructura se ha diseñado utilizando **Docker** apoyado por **Nginx**.

- **Contenerización con Docker:** Docker permite empaquetar el frontend, el backend y la base de datos en contenedores aislados. Cada contenedor incluye sus propias dependencias, librerías y versiones de lenguaje (por ejemplo, el intérprete exacto de PHP). Esto garantiza que el entorno sea 100% reproducible, simplificando el trabajo en equipo y el despliegue final en la nube. Todo el sistema se orquesta mediante un archivo `docker-compose.yml`, que levanta todos los servicios interconectados con un solo comando.
- **Nginx como Servidor Web:** Nginx actúa como la puerta de entrada a la aplicación. Por un lado, sirve de manera altamente eficiente los archivos estáticos generados por React en el frontend. Por otro lado, funciona como un *Proxy Inverso* (Reverse Proxy), interceptando las peticiones dirigidas a la API y redirigiéndolas al contenedor de PHP de forma segura, balanceando la carga y gestionando los encabezados HTTP necesarios para resolver problemas de CORS (Cross-Origin Resource Sharing) entre la interfaz y el servidor.

5. Modelado

5.1. Diagrama entidad-relacion

El Diagrama Entidad-Relación (DER) detalla la arquitectura de la base de datos relacional sobre la que se asienta GymTrack Pro (implementada en MySQL/MariaDB). Dado que el objetivo principal de la aplicación es registrar el progreso y el volumen de entrenamiento de forma precisa, el modelado de datos debe evitar la redundancia y garantizar la integridad referencial.

Las principales entidades identificadas y sus relaciones son las siguientes:

- **Usuarios (users):** Almacena la información de autenticación y perfil del deportista (ID, nombre, correo, contraseña encriptada y fecha de registro). Un usuario puede tener múltiples rutinas y múltiples sesiones de entrenamiento (Relación 1:N).
- **Rutinas (routines):** Estructuras de entrenamiento creadas por un usuario. Contiene datos como el nombre de la rutina (ej. "Empuje", "Pierna pesada") y su descripción.
- **Ejercicios (exercises):** Catálogo maestro de ejercicios disponibles en el sistema (ej. "Press de banca", "Sentadilla"). Contiene el nombre y el grupo muscular implicado.
- **Rutina_Ejercicio (routine_exercises):** Tabla intermedia que rompe la relación Muchos a Muchos (N:M) entre Rutinas y Ejercicios. Permite asociar varios ejercicios a una misma rutina, definiendo el orden en el que se deben ejecutar.
- **Sesiones (sessions):** Representa un entrenamiento físico real en una fecha y hora concretas, asociado a un usuario y, opcionalmente, basado en una rutina preestablecida.
- **Series/Registros (sets_logs):** Es la entidad más volátil e importante para el análisis de la sobrecarga progresiva. Registra los datos de cada serie individual dentro de una sesión: a qué ejercicio pertenece, el peso levantado (kg) y el número de repeticiones alcanzadas.

5.2. Diagrama de casos de uso

El Diagrama de Casos de Uso, perteneciente al estándar UML (Lenguaje Unificado de Modelado), define el comportamiento del sistema desde el punto de vista del usuario, estableciendo las fronteras de la aplicación y documentando las interacciones permitidas según el rol del actor.

En el MVP actual de GymTrack Pro, se define principalmente un actor físico: el **Deportista / Usuario Registrado** (con miras a implementar un rol de *Administrador/Entrenador* en el futuro). Las funcionalidades principales que el sistema ofrece al actor son:

1. **Gestión de cuenta:** Incluye los casos de uso de *Registro de nuevo usuario* y *Autenticación (Login)*. Son procesos obligatorios, ya que la aplicación requiere que los datos de progreso sean estrictamente privados y personalizados.
2. **Gestión de Rutinas (CRUD):** El actor puede *Crear*, *Visualizar*, *Editar* y *Eliminar* planes de entrenamiento. Como caso de uso extendido (<<extend>>), al crear una rutina, el usuario puede *Añadir ejercicios* del catálogo.
3. **Ejecución de Sesión:** Es el núcleo interactivo. El usuario puede *Iniciar un entrenamiento*, lo que desencadena casos de uso iterativos como *Anotar peso y repeticiones por serie* y, finalmente, *Finalizar sesión*.
4. **Análisis de Progreso:** El actor puede *Consultar el Dashboard* interactivo, donde el sistema invoca al motor de visualización para *Generar gráficas de volumen y cargas* basadas en el historial.

5.3. Diagrama de flujo de la aplicación

Mientras que los casos de uso muestran "qué" se puede hacer, el Diagrama de Flujo (o diagrama de actividad) describe el "cómo" y el "cuándo", mapeando el recorrido paso a paso (el *User Journey*) que experimenta el usuario al interactuar con las pantallas del Frontend (React).

El flujo principal de la aplicación está diseñado con una mentalidad *Mobile-First*, buscando la menor cantidad de clics posibles para llegar al registro del entrenamiento:

1. **Punto de entrada:** El usuario accede a la web. El sistema verifica si existe un token de sesión activo. Si no existe, se le redirige al flujo de **Login/Registro**.
2. **Dashboard Principal:** Una vez autenticado, la pantalla inicial muestra un resumen de las métricas recientes y un botón de llamada a la acción primario: *"Iniciar Entrenamiento"*.
3. **Selección y Ejecución:** Al iniciar un entrenamiento, la aplicación transita a la pantalla de selección de rutinas. Tras elegir una, la interfaz cambia al "Modo Entreno". Aquí, el flujo se vuelve cíclico: el usuario navega ejercicio por ejercicio, introduciendo los datos (peso y repeticiones) para cada serie en tiempo real.
4. **Cierre y Retroalimentación:** Al hacer clic en *"Finalizar Entrenamiento"*, el Frontend emite un *payload* masivo a la API de PHP, la base de datos se actualiza y el usuario es devuelto al Dashboard, donde las gráficas de progreso se actualizan inmediatamente para reflejar el volumen recién levantado, ofreciendo un refuerzo psicológico positivo.

6. Recursos

6.1. Recursos Hardware

El desarrollo de una aplicación web moderna, especialmente cuando implica la ejecución simultánea de múltiples contenedores y servidores locales, requiere de un equipo informático con capacidad de procesamiento y memoria suficientes. Los recursos de hardware empleados se dividen en dos entornos:

A. Entorno de Desarrollo (Local): Para la programación y compilación del código se ha utilizado una estación de trabajo principal (ordenador personal) con las siguientes características técnicas aproximadas, necesarias para garantizar fluidez:

- **Procesador (CPU):** Procesador multinúcleo (arquitectura x64), indispensable para el manejo ágil de las máquinas virtuales de Docker y la compilación de los módulos de React.
- **Memoria RAM:** Un mínimo de 16 GB de memoria RAM. El uso intensivo del navegador con herramientas de desarrollador, el IDE, el servidor local de PHP y el motor de base de datos requieren una alta disponibilidad de memoria volátil.
- **Almacenamiento:** Disco de Estado Sólido (SSD), crucial para reducir los tiempos de lectura/escritura en las bases de datos locales y acelerar la instalación de dependencias de Node.js (NPM).

B. Entorno de Pruebas y Despliegue:

- **Dispositivos Móviles (Smartphones):** Dado que la aplicación está diseñada con una filosofía *Mobile-First*, se han utilizado terminales móviles reales (sistemas operativos Android e iOS) para realizar las auditorías de usabilidad, comprobar los gestos táctiles y verificar la adaptabilidad (*responsiveness*) de la interfaz en las instalaciones de un gimnasio.
- **Servidor de Producción (VPS - Virtual Private Server):** Para el despliegue final en la red, se contempla el uso de un servidor privado virtual alojado en la nube. Los requisitos mínimos exigidos para alojar los contenedores Docker de GymTrack Pro son 2 vCPUs, 2 GB de RAM y sistema operativo Linux (Ubuntu/Debian), lo que supone un coste de mantenimiento infraestructural sumamente bajo.

6.2. Recursos Software

El ecosistema de software elegido para este proyecto está compuesto en su totalidad por herramientas gratuitas o de uso libre, seleccionadas por su estándar en la industria profesional del desarrollo web:

- **Entorno de Desarrollo Integrado (IDE):** *Visual Studio Code (VS Code)*. Se ha utilizado como editor de código principal gracias a su ligereza, su terminal integrada y su ecosistema de extensiones (resaltado de sintaxis, formateadores de código y *linters*).
- **Control de Versiones y Repositorio:** *Git* instalado en el sistema local para el control de cambios atómicos, y la plataforma *GitHub* como repositorio remoto en la nube, garantizando que el código fuente esté siempre respaldado y documentado.
- **Infraestructura y Contenedores:** *Docker Desktop* / *Docker Engine* junto con *Docker Compose*. Herramientas vitales para orquestar los contenedores de Nginx, PHP y MySQL sin ensuciar el sistema operativo host.
- **Gestión de Base de Datos:** *phpMyAdmin* o clientes SQL de escritorio (como *DBeaver* o *MySQL Workbench*), utilizados para el diseño del modelo relacional, la administración directa de tablas y la ejecución de *scripts* de migración.
- **Navegadores Web:** *Google Chrome* y *Mozilla Firefox*, haciendo uso intensivo de sus *Developer Tools* (Herramientas para Desarrolladores) para la depuración de JavaScript, análisis del árbol DOM, monitorización de las peticiones de red (API REST) y simulación de dispositivos móviles.
- **Herramientas de Diseño y Modelado:** Plataformas como *Figma* para la creación de esquemas de interfaz (Wireframes) de forma visual, y software de diagramación (como *draw.io* o *Lucidchart*) para la generación de los diagramas UML (Casos de uso y Entidad-Relación) presentados en este documento.

6.3. Recursos Humanos

Detrás del desarrollo de un software de esta envergadura existe un esfuerzo humano considerable en términos de horas de investigación, planificación, codificación y pruebas. En el contexto de este Proyecto Integrado, los recursos humanos involucrados han sido los siguientes:

- **Desarrollador y Gestor del Proyecto (Diego Cabello Bautista):** En calidad de alumno del IES Politécnico Hermenegildo Lanz, asumiendo un rol de perfil *Full-Stack*. Las tareas realizadas abarcan todo el ciclo de desarrollo:

- *Análisis y Gestión*: Toma de requisitos, definición del problema, establecimiento del alcance del MVP y planificación de los tiempos.
- *Diseño (UI/UX)*: Creación de la interfaz gráfica orientada a la usabilidad deportiva.
- *Frontend*: Programación de la lógica del cliente utilizando React.js.
- *Backend y DBA*: Diseño de la base de datos MySQL y programación de los controladores y modelos en PHP.
- *DevOps*: Configuración de contenedores y rutinas de despliegue.
- **Docentes / Tutores del Proyecto Integrado**: Profesorado del instituto encargado de la validación del proyecto, resolución de dudas arquitectónicas puntuales y evaluación de las competencias técnicas adquiridas durante el ciclo formativo.
- **Usuarios de Pruebas (Beta Testers)**: Aunque de forma informal, el proyecto ha contado con la inestimable colaboración de compañeros del entorno del fitness. Estos usuarios reales han interactuado con la aplicación en un entorno de gimnasio durante la "Fase de Pruebas", aportando *feedback* crítico sobre fallos de diseño, errores de lógica en el registro de pesos y sugiriendo mejoras de comodidad visual que se han ido integrando en el producto final.

7. Recursos

La gestión temporal de GymTrack Pro se ha regido por principios de metodologías ágiles (inspiradas en Scrum), adaptadas al trabajo individual. En lugar de seguir un modelo tradicional en cascada (Waterfall), el proyecto se dividió en iteraciones o *sprints* de una a dos semanas, permitiendo tener versiones funcionales del MVP de forma progresiva.

1. **Fase 1: Análisis y Modelado (Semanas 1-2)**: Definición de requisitos, diseño de la base de datos y creación de los *wireframes* de la interfaz gráfica en Figma.
2. **Fase 2: Infraestructura y Backend (Semanas 3-4)**: Configuración de los contenedores Docker, creación del esquema relacional en MySQL y programación de los *endpoints* principales de la API REST en PHP (Autenticación y CRUD de rutinas).
3. **Fase 3: Desarrollo Frontend (Semanas 5-7)**: Configuración del proyecto React, enrutamiento, diseño *Mobile-First* y conexión con la API para el consumo de datos.
4. **Fase 4: Integración y Motor de Gráficas (Semana 8)**: Implementación de la lógica matemática para el cálculo de volumen y renderizado de las gráficas de progreso.
5. **Fase 5: Testing y Depuración (Semanas 9-10)**: Fase intensiva de pruebas de usabilidad, corrección de *bugs* (debugging) en el registro de series y optimización del rendimiento.

8. Recursos

8.1. Diseño de la Base de Datos

La implementación física de la base de datos se llevó a cabo en MySQL. Se establecieron restricciones de integridad referencial (FOREIGN KEY) y políticas de borrado en cascada (ON DELETE CASCADE) para evitar registros huérfanos. Por ejemplo, si un usuario elimina su cuenta, todas sus rutinas y sesiones asociadas se eliminan automáticamente.

Para el diseño de las tablas, se utilizaron tipos de datos óptimos: VARCHAR para cadenas de texto limitadas (correos, contraseñas hashadas), INT para identificadores y repeticiones, y DECIMAL(5,2) para los pesos, permitiendo registrar incrementos de carga de hasta gramos (ej. 22.50 kg).

8.2. Arquitectura del backend (API REST en PHP)

El backend de GymTrack Pro no renderiza HTML, sino que actúa exclusivamente como un proveedor de datos mediante una API RESTful. El código PHP se ha estructurado siguiendo el patrón MVC (Modelo-Vista-Controlador), aunque adaptado a la arquitectura de una API (donde la "Vista" es simplemente una respuesta JSON).

Se ha utilizado **PDO** (PHP Data Objects) para la conexión a la base de datos, asegurando la prevención de inyecciones SQL mediante el uso de sentencias preparadas (*Prepared Statements*). Cada petición HTTP entrante es procesada por un enrutador (Router) central, que deriva la solicitud al controlador adecuado (UserController, RoutineController, etc.) dependiendo del verbo HTTP (GET, POST, PUT, DELETE) y la URL.

8.3. Implementación del frontend con React.js

El frontend se ha construido como una SPA (Single Page Application) utilizando **React.js**. Para la navegación interna sin recarga de página, se implementó React Router. El estado global de la aplicación (como la información del usuario logueado o el modo oscuro/claro) se ha manejado mediante la API de Contexto (Context API) nativa de React, evitando la sobrecarga de dependencias externas como Redux.

El diseño se enfocó en crear componentes reutilizables (Botones, Tarjetas de Ejercicio, Modales). Para los estilos, se ha empleado CSS moderno, garantizando que la interfaz sea 100% *Responsive*, con un diseño orientado a botones grandes e interacciones táctiles (*Mobile-First*) para facilitar su uso con las manos sudorosas en medio de un entrenamiento.

8.4. Configuración de Docker y Nginx

Para asegurar que el entorno sea idéntico en cualquier máquina, se configuró un archivo `docker-compose.yml` que orquesta tres contenedores principales:

1. **Contenedor Web (Nginx):** Actúa como proxy inverso. Escucha en el puerto 80 (HTTP) y sirve los archivos estáticos de React. Si la petición empieza por `/api`, Nginx redirige el tráfico internamente al contenedor de PHP.
2. **Contenedor Backend (PHP-FPM):** Ejecuta el intérprete de PHP y procesa la lógica de la aplicación.
3. **Contenedor Base de Datos (MySQL):** Almacena los datos. Se mapearon volúmenes de Docker para asegurar que la información no se pierda si el contenedor se reinicia.

8.5. Sistema de autenticación con JWT

Para mantener el sistema sin estado (*Stateless*), característico de las APIs REST, se ha implementado un sistema de autenticación basado en **JSON Web Tokens (JWT)**. Cuando un usuario hace *Login* correctamente, el backend de PHP genera un token cifrado y firmado digitalmente que contiene el ID del usuario y una fecha de expiración. Este token se devuelve a React, que lo almacena de forma segura en el almacenamiento local (*LocalStorage*). A partir de ese momento, cada vez que React solicita datos privados a la API, inyecta este JWT en la cabecera `HTTP Authorization: Bearer <token>`. El servidor PHP verifica la firma del token antes de procesar la petición, garantizando la seguridad.

8.6. Motor de gráficas y visualización de datos

El valor diferencial de GymTrack Pro reside en transformar datos crudos en información útil. En el frontend, se integró una librería de visualización de datos. La lógica de implementación requiere que el backend sume el volumen total levantado por sesión (Fórmula: *Peso x Repeticiones x Series*) y devuelva un arreglo de datos cronológicos. React recibe estos datos y los mapea en un componente de gráfica lineal, permitiendo al deportista ver su curva de progresión en un ejercicio específico a lo largo de los meses.

9. Fase de pruebas

9.1. Estrategia de pruebas

Se adoptó una estrategia de pruebas combinada: validaciones automatizadas para la lógica de negocio pura y pruebas manuales exhaustivas para la interfaz gráfica y la experiencia de usuario (UX).

9.2. Pruebas unitarias del backend

En el código PHP se testearon funciones aisladas. Por ejemplo, se comprobó que el algoritmo de cálculo de volumen total manejara correctamente valores nulos, decimales, o entradas de texto maliciosas, asegurando que las matemáticas subyacentes del entrenamiento fueran siempre exactas.

9.3. Pruebas de integración de la API

Utilizando herramientas como **Postman**, se simulaban peticiones HTTP a todos los *endpoints* de la API. Se verificó que:

- Una petición de borrado (DELETE) a una rutina ajena devuelva un error HTTP 403 (Prohibido).
- La creación de un usuario con un email ya existente devuelva un código HTTP 409 (Conflicto).
- Las respuestas JSON tengan la estructura exacta que el frontend de React espera recibir.

9.4. Pruebas de interfaz y usabilidad

Se realizaron despliegues locales y se accedió a la plataforma desde dispositivos Android e iOS. Se comprobó:

- Que los modales y teclados numéricos de los smartphones no oculten los botones de confirmación.
- El correcto funcionamiento de las alertas visuales (*Toast notifications*) al guardar una serie con éxito.

9.5. Pruebas de rendimiento y carga

El rendimiento en el frontend se auditó utilizando **Google Lighthouse**. Se optimizó el tamaño de los recursos estáticos y se confirmó que el *Virtual DOM* de React gestiona eficientemente el re-renderizado sin "congelar" la pantalla, incluso cuando se editan rutinas con decenas de ejercicios.

9.6. Pruebas de seguridad

Se aplicaron las directrices básicas de seguridad OWASP:

- Prevención de inyecciones SQL mediante PDO.
- Mitigación de ataques XSS (Cross-Site Scripting) gracias a que React escapa automáticamente el texto renderizado en el DOM.
- Hasheo seguro de contraseñas en la base de datos utilizando algoritmos modernos (PASSWORD_BCRYPT en PHP).

10. Documentación

La transferibilidad y el mantenimiento futuro del proyecto dependen de una buena documentación. Para GymTrack Pro, se han elaborado los siguientes recursos:

1. **Código comentado y limpio:** Siguiendo las convenciones de nombres (*CamelCase* en JavaScript, *snake_case* en la base de datos) y comentando las funciones más complejas.

2. **Archivo README.md:** Ubicado en la raíz del repositorio de GitHub, proporciona instrucciones detalladas sobre cómo clonar el proyecto, cómo levantar los contenedores con Docker y cuáles son las variables de entorno (.env) necesarias.
3. **Documentación de la API:** Un documento técnico que detalla todas las rutas de la API, los parámetros requeridos y ejemplos de respuestas esperadas, facilitando la integración a futuros desarrolladores.

11. Conclusión

La realización del proyecto **GymTrack Pro** ha supuesto la culminación práctica y profesional de los conocimientos adquiridos a lo largo del ciclo formativo. El desarrollo de esta aplicación web no solo ha representado un reto a nivel técnico, sino que ha exigido una profunda labor de análisis, planificación y comprensión de las necesidades reales de un sector en constante crecimiento como es el del fitness y el entrenamiento de fuerza.

A continuación, se exponen las conclusiones finales derivadas de las distintas fases del proyecto, evaluando el grado de cumplimiento de los objetivos, los retos tecnológicos superados y las perspectivas de futuro de la aplicación.

Consecución de Objetivos y Resultados Obtenidos El objetivo principal de este proyecto era diseñar y desarrollar una plataforma capaz de solucionar un problema muy extendido entre los deportistas: la falta de un registro estricto y cuantificable de la sobrecarga progresiva. Tras la fase de implementación y pruebas, se puede afirmar que GymTrack Pro cumple satisfactoriamente con los requisitos funcionales establecidos para su Producto Mínimo Viable (MVP).

Se ha logrado entregar un sistema robusto donde los usuarios pueden registrarse de forma segura, gestionar sus propias rutinas mediante un constructor dinámico e intuitivo, y anotar sus marcas (pesos y repeticiones) en tiempo real durante sus sesiones. Además, el motor de visualización de datos ha transformado con éxito la información en bruto en gráficas comprensibles, otorgando al usuario una herramienta real para evitar el estancamiento y tomar decisiones basadas en datos.

Evaluación Técnica y Arquitectura A nivel tecnológico, la elección del stack ha demostrado ser acertada y eficiente:

- **Frontend (React.js):** El uso de React ha permitido construir una Interfaz de Usuario (UI) altamente reactiva y modular. La gestión del estado ha sido clave para que el usuario experimente una navegación fluida, similar a la de una aplicación nativa, algo indispensable cuando se utiliza la aplicación desde un dispositivo móvil en medio de un gimnasio.

- **Backend (PHP) y Base de Datos (MySQL/MariaDB):** La lógica de negocio y la gestión de la base de datos relacional han garantizado la integridad, seguridad y persistencia de la información. El diseño de la base de datos se ha optimizado para manejar un volumen creciente de registros por usuario sin comprometer los tiempos de respuesta.
- **Infraestructura (Docker y Nginx):** Una de las decisiones arquitectónicas más valiosas ha sido la contenerización de la aplicación. Gracias a Docker, el despliegue de GymTrack Pro es replicable, escalable y libre de los clásicos problemas de compatibilidad entre entornos de desarrollo y producción.

Dificultades y Retos Superados A lo largo del ciclo de vida del desarrollo, surgieron diversas dificultades inherentes a un proyecto de esta envergadura. Uno de los mayores retos fue lograr un diseño verdaderamente responsive y adaptado a móviles (*Mobile-First*). El entorno del gimnasio exige que la interacción con la pantalla sea rápida, con botones accesibles y sin distracciones; adaptar tablas complejas de rutinas a pantallas pequeñas requirió un esfuerzo considerable de diseño UI/UX. Asimismo, la integración fluida entre el cliente (React) y el servidor (PHP), asegurando una autenticación robusta, supuso una curva de aprendizaje importante, consolidando conceptos de seguridad informática, protección contra inyecciones SQL y ataques XSS o CSRF.

Perspectivas y Líneas de Trabajo Futuro GymTrack Pro ha sido concebido bajo una arquitectura escalable, lo que deja la puerta abierta a una amplia gama de mejoras continuas. Como se definió en el análisis inicial, las fases posteriores del desarrollo contemplan la incorporación de funcionalidades que aportarán un valor añadido significativo:

- La implementación de un sistema social que permita conectar con amistades, puntuar rutinas mediante estrellas y dejar comentarios.
- La inclusión de una calculadora de 1RM predictiva, así como la generación de informes exportables en PDF o CSV.
- El seguimiento de medidas corporales, la vinculación con listas de reproducción de música y un calendario inteligente con alertas de inactividad.
- La futura distinción entre "Usuario Normal" y "Entrenador", dotando a los preparadores físicos de un panel de control para gestionar y asignar rutinas a sus clientes.

Valoración Personal A nivel personal y académico, como alumno del IES Politécnico Hermenegildo Lanz, este Proyecto Integrado ha sido la oportunidad perfecta para simular el ciclo de trabajo de una empresa de desarrollo de software real. Ha exigido disciplina en la planificación, autonomía para la resolución de errores (debugging) y una visión global que abarca desde la primera línea de código hasta el despliegue en un servidor. El desarrollo de GymTrack Pro me ha permitido no solo demostrar mi competencia técnica en el apilamiento de tecnologías (Frontend, Backend, Bases de Datos y DevOps), sino también mi capacidad para conceptualizar una necesidad del mundo real y

traducirla en una solución de software funcional, atractiva y escalable. Cierro esta etapa formativa con un producto del cual me siento profundamente orgulloso y que sienta unas bases sólidas para mi futura carrera como desarrollador.

12. Bibliografía

Para el desarrollo de este proyecto se han consultado fuentes oficiales de documentación tecnológica, así como bibliografía relacionada con las ciencias del deporte para la correcta aplicación teórica de la sobrecarga progresiva.

Recursos Tecnológicos:

- **React Docs (Meta):** Documentación oficial sobre la gestión de estado y ciclos de vida de componentes. (<https://react.dev/>)
- **PHP Official Manual:** Referencias sobre PDO, seguridad de contraseñas y peticiones HTTP. (<https://www.php.net/manual/es/>)
- **Docker Documentation:** Guías de contenerización y configuración de docker-compose. (<https://docs.docker.com/>)
- **MySQL Reference Manual:** Optimización de consultas y diseño relacional. (<https://dev.mysql.com/doc/>)
- **MDN Web Docs (Mozilla):** Estándares web, CSS Flexbox/Grid y JavaScript ES6+. (<https://developer.mozilla.org/>)

Recursos Teóricos (Ciencias del Deporte):

- Schoenfeld, B. J. (2010). *The Mechanisms of Muscle Hypertrophy and Their Application to Resistance Training*. Respaldo teórico sobre la importancia del volumen y la sobrecarga progresiva.
- Rippetoe, M. (2011). *Starting Strength: Basic Barbell Training*. Referencia para la estructuración de rutinas de fuerza y progresión de cargas aplicadas al MVP.